

Business Requirements Document

AI Changelog & Release Notes Generator (“ChangeLogScriptGenerator”)

Owner: Srikanth Pusapati · Version 1.0 · July 31, 2026 · Status: Draft

1. Executive Summary

Small development teams and solo builders ship software constantly but rarely communicate it well. Writing customer-facing changelogs and release notes is pure toil: it is repetitive, low-status work that gets skipped, done badly, or done at 11 PM before a launch. This project builds a tool that converts raw engineering activity (Git commits, merged pull requests, and issue references) into polished, audience-appropriate changelogs and release announcements using an LLM pipeline.

The product starts as a free CLI and GitHub Action (community distribution, zero infrastructure), and graduates into a hosted SaaS with team plans at \$15–29/month once validated. The primary strategic goal is durable, compounding side income built at 5–15 hours/week, with the developer community (friends, colleagues, open-source maintainers, HN/Reddit/Discord) as the first distribution channel.

2. Problem Statement

- Release notes are written manually from memory or by skimming git log, taking 30–90 minutes per release and often shipping late or not at all.
- Raw commit messages are written for engineers (“fix null check in webhook retry”), not customers (“Webhooks now retry reliably”). Translating between the two audiences is the actual hard part.
- Existing options are either heavyweight product-marketing suites, or naive “dump commits into a template” scripts that produce output nobody would publish.
- Teams that do publish changelogs consistently see measurable gains in user trust, feature adoption, and reduced “what changed?” support tickets — the value is known, the toil is the blocker.

3. Goals, Success Metrics & Kill Criteria

3.1 Goals

- **G1 — Ship fast:** a working CLI MVP within 2 weekends; a GitHub Action within 4 weeks.
- **G2 — Validate demand:** 20+ email signups or 10+ active repos using the free tool within 6 weeks of launch.
- **G3 — Monetize:** first paying team within 3 months of the hosted version going live.

- **G4 — Learn:** use this build to master LLM engineering patterns (structured output, evals, agentic workflows) transferable to future AI-for-SMB products.

3.2 Success Metrics

Metric	Target	Measured by
Time-to-changelog for a release	< 2 minutes (vs 30–90 min manual)	User feedback / demo timing
Output acceptance rate	≥ 80% of generated entries published with light edits	Diff between generated and published
Free tool adoption	20+ signups or 10+ repos in 6 weeks	GitHub installs, landing page emails
Community engagement	1 useful launch thread (HN/Reddit) + 5 friends using it	Manual tracking
Revenue (Phase 3)	First paying team in 90 days of SaaS launch	Stripe

3.3 Kill Criteria (pre-committed)

If after 6 weeks of the free tool being public there are fewer than 20 signups AND no one has agreed to pay or pre-order the hosted version, archive the project in PersonalProjects with a written retro and move to the next shortlist idea (Webhook Observability Dashboard). Do not extend the deadline; extend only if there is clear qualitative pull (users asking for the hosted version unprompted).

4. Target Users & Personas

Persona	Context	Pain	What they buy
Indie hacker / solo founder	Ships weekly, markets alone	No time; changelog is marketing they skip	One-command polished release notes
Small SaaS team lead (2–10 devs)	Biweekly releases, has customers	Devs won't write; PM rewrites badly	Consistent, on-brand changelog with approval flow
Open-source maintainer	Tags releases on GitHub	Release notes are raw commit dumps	Free Action that writes human release notes
DevRel / PM (Phase 3)	Owns comms for a product	Chasing engineers for 'what shipped'	Auto-drafted announcements from merged PRs

5. Scope

5.1 In Scope — Phase 1 (CLI MVP)

- Collect commits and merged PRs between two Git refs (tag-to-tag, or last N days).
- Classify changes: Features, Improvements, Bug Fixes, Breaking Changes; filter internal noise (chore, ci, test).
- LLM rewrite into customer-facing language with configurable tone (professional / friendly / technical) and audience.
- Render Markdown (Keep-a-Changelog style) and GitHub Release format; write to file or stdout.
- Config file (.changelog.yml) for tone, audience, sections, and ignore rules.

5.2 In Scope — Phase 2 (GitHub Action)

- Trigger on tag push; open a PR updating CHANGELOG.md and/or publish GitHub Release notes automatically.
- Human-in-the-loop by default: output arrives as a PR for review, never a silent commit.

5.3 In Scope — Phase 3 (Hosted SaaS, post-validation)

- GitHub App with dashboard, hosted public changelog pages, Slack/email announcements, team styles, Stripe billing.

5.4 Out of Scope (for now)

- Jira/Linear ingestion (Phase 3+ backlog), GitLab/Bitbucket support, in-app changelog widgets, localization, mobile apps.

6. Functional Requirements

ID	Requirement	Priority	Phase
FR-1	Collect commits + merged PR titles/bodies/labels between two refs via git and the GitHub API	Must	1
FR-2	Parse Conventional Commits prefixes (feat/fix/chore...) when present; degrade gracefully when absent	Must	1
FR-3	Classify each change into Features / Improvements / Fixes / Breaking; drop internal-only changes	Must	1
FR-4	Generate customer-facing prose via LLM with configurable tone and audience	Must	1
FR-5	Enforce structured LLM output (JSON schema) and validate before rendering; retry once on invalid output	Must	1
FR-6	Render Markdown changelog + GitHub Release notes; idempotent re-runs	Must	1
FR-7	Read settings from .changelog.yml (tone, audience, sections, ignore globs, breaking-change labels)	Must	1
FR-8	Run as GitHub Action on tag push; open PR with generated changelog	Must	2
FR-9	Publish GitHub Release notes via API on approval	Should	2
FR-10	Redact secrets/internal URLs from prompts before sending to the LLM	Must	1
FR-11	Hosted dashboard, public changelog page, Slack/email digests, billing	Could	3

7. Non-Functional Requirements

Category	Requirement
Performance	End-to-end generation < 60s for a release of ≤ 200 commits.
Cost	LLM cost < \$0.05 per typical release (batch changes into one call; use a mid-tier model).
Privacy	Only commit/PR metadata leaves the machine; never source code diffs by default. Document this clearly — it is a selling point.
Reliability	Deterministic pipeline around a non-deterministic LLM: schema validation, one retry, then fail loudly with raw grouped output as fallback.
Security	API keys via env vars / Action secrets only; never logged. No code execution from LLM output.
Usability	Zero-config first run must produce something publishable on a repo with decent commit messages.

8. Monetization & Pricing

- **Free tier (forever):** CLI + GitHub Action, public repos, community goodwill and distribution engine.
- **Pro — \$15/mo per team:** private repos via hosted app, custom tone/brand voice, hosted changelog page.
- **Team — \$29/mo:** Slack/email announcements, multiple repos, approval workflow, analytics on changelog views.

Rationale: free open-source core is the marketing channel (GitHub Marketplace, HN, dev communities); revenue comes from convenience and hosting, which teams pay for happily once the free tool proves value.

9. Risks & Assumptions

Risk	Likelihood	Mitigation
Low moat — easy to copy	High	Win on distribution (Marketplace, community), polish, and vertical features (approval flow, brand voice). Ship fast; brand beats clone.
Bad commit messages produce bad output	Medium	Lean on PR titles/bodies (usually better), classification prompt tuned for messy input, and clear docs on getting good output.
Teams distrust sending data to an LLM	Medium	Metadata-only by default, bring-your-own-API-key mode, clear privacy doc.
Motivation fade at 5–15 hrs/wk	Medium	Pre-committed kill criteria; small shippable phases; deliver to friends early for feedback dopamine.
GitHub Marketplace/API policy changes	Low	Core is a portable CLI; Action is a thin wrapper.

Assumptions: target users have GitHub-centric workflows; an LLM API key (or ours, metered) is acceptable; English-only output is fine for validation.

10. Validation Plan

- Weeks 1–2: Build CLI MVP; run it on 3 real repos (yours + 2 friends'). Publish output samples.
- Weeks 3–4: GitHub Action + README + landing page with email capture. Hand to 10 dev friends; ask each for one honest complaint.
- Weeks 5–6: Launch threads (Show HN, r/webdev, dev Discords). Track signups/installs. Interview every active user.
- Gate: ≥ 20 signups or 10 active repos $\rightarrow$ build hosted Pro. Otherwise archive with retro per kill criteria.